

Typst Tutorial

School Template Handleiding v1.0

Ruben Ryckaert

2025-2026



KU Leuven
Faculteit Industriële Ingenieurswetenschappen

Disclaimer: Deze samenvattingen zijn beschikbaar op GitHub.
Draag gerust bij: [GitHub Repository](#)

Inhoudsopgave

1. Snel aan de slag	5
1.1. Je eerste document	5
1.2. Verschil met LaTeX	5
1.3. VS Code Setup	5
2. Basis Typst Syntax	6
2.1. Tekstopmaak	6
2.2. Tekstgrootte	6
2.3. Lijstjes maken	7
2.4. Nieuwe regel of nieuwe alinea?	7
2.5. Commentaar	7
3. Tekst Markeren & Annoteren (mannot)	7
3.1. Markeren	8
3.2. Annotaties	8
3.3. Positionering	8
3.4. Pijlen (Leader Lines)	9
3.5. Meerdere Elementen	9
3.6. Geavanceerd: CeTZ Annotaties	9
4. Samenwerken en Issues (GitHub)	9
4.1. Issues Aanmaken	9
4.2. Aan een Issue Werken	10
4.3. Klaar? Pull Request!	10
5. Eenheden en Getallen (unify)	10
5.1. Basis gebruik	10
5.2. Bereiken	10
6. Document Structuur & Layout	11
6.1. Pagina-instellingen	11
6.2. Kolommen	11
6.3. Inhoudsopgave (Outline)	11
6.4. Bibliografie & Referenties	11
7. Box Omgevingen	12
7.1. Concept (Definities)	12
7.2. Theorie (Stellingen)	12
7.3. Oefening (Oefeningen)	13
7.4. Voorbeeld (Praktijkvoorbeelden)	13
7.5. Waarschuwing (Warningbox)	13
7.6. Theorem	13
7.7. Alle box types overzicht	13

8. Formules met #frm	14
8.1. Gebruik	14
8.2. Meer voorbeelden	14
8.3. Text wrap rond figuren	15
9. Afbeeldingen Toevoegen (Nieuwe Workflow)	15
9.1. Snippets	16
9.2. Formularium afdrukken	16
10. Symbolen met #sym	16
10.1. Eerste vermelding	16
10.2. Herhaald gebruik	16
10.3. Symbolenlijst afdrukken	16
11. Wiskunde in Typst	17
11.1. Basis wiskunde	17
11.2. Verzamelingen (Math Helpers)	17
11.3. Afgeleiden (physica pakket)	17
11.4. Matrices en Vectoren	18
11.5. Stelsels en Uitlijning	18
11.6. Veelgebruikte symbolen	18
11.6.1. Griekse letters	18
11.6.2. Pijlen	19
11.6.3. Operatoren en Relaties	19
11.6.4. Logica en Verzamelingen	19
12. Cross-References	20
13. Figuren	20
13.1. Standaard Figuur	20
13.2. Figuur naast tekst	20
14. Tabellen	21
15. Grafieken en Plots (cetz-plot)	21
15.1. Functieplot	22
15.2. Trigonometrische functies	22
15.3. Scatterplot (data plotten)	23
15.4. Staafdiagram (Bar chart)	23
16. Extra Grafieken (Pie & Histogram)	24
16.1. Histogram (Bar Chart)	24
16.2. CeTZ Tekeningen (TikZ equivalent)	24
17. Code Listings	25
18. Veelgemaakte Fouten	25
19. Utility Macro's	26
19.1. Workflow markers	26
19.2. Examentip	26
19.3. Belangrijke termen markeren	26
19.4. Kolommen	26
20. LaTeX vs Typst Vergelijking	26

21. Inbegrepen Pakketten (Typst Universe)	27
21.1. unify (siunitx equivalent)	27
21.2. physica (physics math)	27
21.3. CeTZ (TikZ equivalent)	28
21.4. Andere handige pakketten	29
21.4.1. Fletcher — Diagrammen met pijlen	29
21.4.2. Algo — Pseudocode en Algoritmes	30
21.4.3. Glossarium — Woordenlijst	31
21.4.4. Tablem — Markdown-achtige tabellen	31
22. Afbeeldingen naast elkaar	31
23. Customisatie (Set & Show Rules)	32
23.1. Set Rules	32
23.2. Show Rules	32
24. Scripting & Automatisatie	33
24.1. Variabelen en Functies	33
24.2. Conditionals (If/Else)	33
24.3. Lussen (Loops)	33
25. Geavanceerde CeTZ Voorbeelden	33
25.1. Taylor Reeks (Fill Between)	34
25.2. 3D Orthogonale Projectie	35
25.3. Pie Chart (Landen)	35
25.4. Pyramid Chart (Financieel)	36
Symbolenlijst	38
Formularium	38

1. Snel aan de slag

1.1. Je eerste document

Een minimaal Typst document met school-template ziet er als volgt uit:

```
Minimaal Voorbeeld

#import "school-template.typ": *

#show: project.with(
  title: "Mijn Samenvatting",
  course: "Vaknaam",
  authors: ("Jouw Naam"),
  academic_year: "2025-2026",
)

= Inleiding
Hier begint je inhoud...

// Appendix
#printsymbols()
#printformulairium()
```

⚠ Typst vs LaTeX

Typst compileert **veel sneller** dan LaTeX en heeft een eenvoudigere syntax. Je hoeft geen extra compilatieronden te doen — alles wordt in één keer verwerkt. Typst is ook **live-preview** in VS Code met de Tinymist extensie.

1.2. Verschil met LaTeX

📄 Waarom Typst?

Voordelen van Typst:

- Veel snellere compilatie (milliseconden in plaats van seconden)
- Eenvoudigere, leesbaardere syntax
- Ingebouwde scripting (geen aparte pakketten nodig)
- Live preview in VS Code

Beperkingen:

- Kleiner ecosysteem dan LaTeX (minder pakketten)
- Geen `tikz` of `circuitikz` (gebruik afbeeldingen in plaats daarvan)
- Nog relatief nieuw, sommige features ontbreken

1.3. VS Code Setup

Om met Typst te werken in VS Code heb je de **Tinymist** extensie nodig. Deze geeft je:

- Live preview van je document
- Syntax highlighting
- Autocomplete voor Typst functies

💡 Installatie

1. Installeer de **Tinymist** extensie in VS Code
2. Open een .typ bestand
3. Klik op het preview icoon rechtsboven (of `Ctrl+K V`)
4. Je ziet nu een live preview van je document

2. Basis Typst Syntax

2.1. Tekstopmaak

Typst heeft een eenvoudige syntax voor tekstopmaak:

```

Basis Tekstopmaak

"Vetgedrukt"
_Cursief_
#underline[Onderstreept]
#emph[Nadruk]
'monospace'
#text(fill: red)[Gekleurde tekst]
#smallcaps[Kleine Hoofdletters]

= Heading 1
== Heading 2
=== Heading 3

```

Resultaat:

Vetgedrukt

Cursief

Onderstreept

Nadruk

monospace

Gekleurde tekst

KLEINE HOOFDLETTERS

2.2. Tekstgrootte

Je kan de grootte van de tekst aanpassen met `#text(size: ...)`:

Code	Resultaat
<code>#text(size: 6pt)[Heel klein]</code>	Heel klein
<code>#text(size: 8pt)[Klein]</code>	Klein
<code>#text(size: 10pt)[Normaal]</code>	Normaal
<code>#text(size: 14pt)[Groot]</code>	Groot
<code>#text(size: 18pt)[Groter]</code>	Groter
<code>#text(size: 24pt)[Nog groter]</code>	Nog groter

2.3. Lijstjes maken

Lijstjes zijn super handig voor samenvattingen!

```
Opsommingen

// Ongeordende lijst (bolletjes)
- Eerste punt
- Tweede punt
- Genest punt

// Geordende lijst (nummers)
+ Stap 1
+ Stap 2
+ Sub-stap
```

Resultaat:

- Eerste punt
 - Tweede punt
 - Genest punt
1. Stap 1
 2. Stap 2
 1. Sub-stap

2.4. Nieuwe regel of nieuwe alinea?

- **Nieuwe alinea:** Laat **één witregel** open in je code. Dit is de standaard manier!
- **Harde enter:** Gebruik `\` aan het einde van een regel.
- **Nieuwe pagina:** Gebruik `#pagebreak()`.

```
Regels en Alinea's

Dit is de eerste alinea.

Dit is een nieuwe alinea (witregel ertussen).

Dit is regel 1 \
Dit is regel 2 (harde enter met backslash)
```

2.5. Commentaar

Alles wat je achter `//` typt, wordt genegeerd door Typst:

```
Commentaar

Dit is tekst die in de PDF komt.
// Dit is een notitie, dit zie je niet in de PDF.
Hier gaat de tekst verder.
```

3. Tekst Markeren & Annoteren (mannot)

De template bevat het **mannot** pakket voor handgeschreven-stijl markeringen en annotaties.

3.1. Markeren

Gebruik de volgende functies om tekst te markeren:

- `#mark`: Verandert de tekstkleur.
- `#markhl`: Markeerstift (highlight) effect.
- `#markrect`: Tekent een rechthoek (kader).
- `#markul`: Onderlijnt de tekst.

```

$
mark(x, color: #red)
+ markhl(f(x))
+ markrect(e^x)
+ markul(x + 1)
$

```

Je kunt kleuren en stijlen aanpassen:

```

$
mark(x, color: #green)
+ markhl(f(x), color: #purple, stroke: 1pt)
+ markrect(e^x, color: #red, fill: #blue, outset: 0.2em)
$

```

3.2. Annotaties

Om tekst te annoteren, moet je het eerst markeren met een unieke tag. Daarna gebruik je `#annot` om de notitie te plaatsen.

```

$
markhl(x, tag: #<1>)
+ markrect(y, tag: #<2>)

#annot(<1>)[Dit is x]
#annot(<2>, pos: top)[Dit is y]
$

```

3.3. Positionering

Gebruik `pos`, `dx` en `dy` om de annotatie te positioneren:

- `pos`: Positie relatief aan de markering (bv. top, bottom, left, right).
- `dx`, `dy`: Verschuiving in x- en y-richting.

```

$
mark(z, tag: #<z>)
#annot(<z>, pos: top + right, dy: -1em)[Annotatie]
$

```

3.4. Pijlen (Leader Lines)

Als de annotatie ver van de tekst staat, wordt automatisch een pijl getekend.

- `leader-connect: "elbow"` maakt een gehoekte pijl.
- `leader-stroke`: Stijl van de lijn (bv. `1pt + red`).
- `leader-tip` / `leader-toe`: Pijlpunten (via `tiptoe` package).

```

$
markrect(t, tag: #<t>)
#annot(<t>, pos: bottom, dy: 2em, leader-connect: "elbow")[Tijd]
$
  
```

3.5. Meerdere Elementen

Je kunt één annotatie naar meerdere elementen laten wijzen door een array van tags mee te geven.

```

$
mark(a, tag: #<a>) + mark(b, tag: #<b>)
#annot(<a>, <b>, pos: top, dy: -1.5em)[Variabelen]
$
  
```

3.6. Geavanceerd: CeTZ Annotaties

Voor complexe tekeningen kun je `#annot-cetz` gebruiken om direct op de gemarkeerde elementen te tekenen met CeTZ.

```

$
mark(x, tag: #<x>)
#annot-cetz(<x>, cetz, {
  import cetz.draw: *
  line("x.north", (0, 2), name: "l")
  content("l.end", [Punt])
})
$
  
```

4. Samenwerken en Issues (GitHub)

Samenwerken aan deze samenvattingen gebeurt via GitHub. We gebruiken de VS Code extensie **GitHub Pull Requests and Issues** om dit makkelijk te maken.

4.1. Issues Aanmaken

Als je een fout vindt of iets wilt toevoegen, kun je direct vanuit je code een “Issue” aanmaken op GitHub.

Stap-voor-stap: Issue maken

1. Typ in je Typst bestand een commentaar met `// @issue:` of `// TODO:`.
2. Klik op het lampje dat verschijnt (of druk `Ctrl+.`).
3. Kies **“Create issue from comment”**.
4. VS Code maakt nu automatisch een taak aan op GitHub.

4.2. Aan een Issue Werken

1. Ga naar de **Source Control** tab (links, met het vertakking-icoon).
2. Kijk onderaan bij **GitHub / Issues**.
3. Rechtsklik op een issue en kies **“Start Working on Issue”**.
4. VS Code maakt automatisch een nieuwe *branch* aan.

4.3. Klaar? Pull Request!

1. Commit je wijzigingen (“Commit & Push”).
2. Er verschijnt een knop **“Create Pull Request”**.
3. In de beschrijving kun je het issue automatisch sluiten: `Fixes #123`.

5. Eenheden en Getallen (unify)

De template bevat het **unify** pakket – het Typst-equivalent van LaTeX’s `siunitx`. Hiermee kun je getallen, eenheden en bereiken correct formatteren.

5.1. Basis gebruik

Code	Resultaat
<code>\$qty("50", "km/h")\$</code>	50 km h ⁻¹
<code>\$qty("9.81", "m/s^2")\$</code>	9.81 m s ⁻²
<code>\$qty("1.5", "kN")\$</code>	1.5 kN
<code>\$qty("3e8", "m/s")\$</code>	3 · 10 ⁸ m s ⁻¹
<code>\$num("1234567")\$</code>	1 234 567
<code>\$num("3.14159")\$</code>	3.141 59
<code>\$num("-1.32865+-0.50273e-6")\$</code>	(−1.328 65 ± 0.502 73) · 10 ⁻⁶

5.2. Bereiken

1 – 10
(20 – 30) m s⁻¹

Code:

• • •
Bereiken

```
$numrange("1", "10")$
$qtyrange("20", "30", "m/s")$
```

⚠ **EXAMENTIP:** Gebruik **altijd** `qty()` of `num()` voor getallen en eenheden om consistente spatiëring en opmaak te krijgen.

6. Document Structuur & Layout

6.1. Pagina-instellingen

Je kan de pagina-afmetingen en marges aanpassen met `#set page(...)`. Meestal staat dit bovenaan je document.

```
Pagina Setup

#set page(
  paper: "a4",
  margin: (x: 2cm, y: 2cm),
  numbering: "1 / 1",
)
```

6.2. Kolommen

Je kan tekst in kolommen zetten met de `columns()` functie of `#show: columns`.

```
Kolommen

// Voor het hele document:
#show: rest => columns(2, rest)

// Of voor een specifiek blok:
#columns(2)[
  Dit is tekst in de eerste kolom.
  #colbreak()
  Dit staat in de tweede kolom.
]
```

6.3. Inhoudsopgave (Outline)

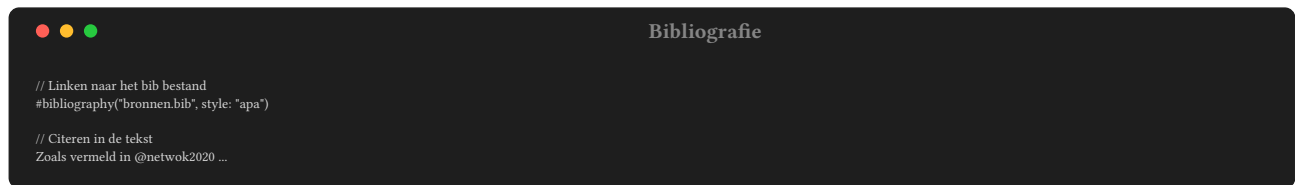
Een inhoudsopgave wordt automatisch gegenereerd op basis van je headings (=, ==, ...).

```
TOC

#outline(
  title: "Inhoud",
  indent: auto,
  depth: 3
)
#pagebreak()
```

6.4. Bibliografie & Referenties

Typst heeft ingebouwde bibliografie-ondersteuning. Upload een `.bib` bestand en link het.



7. Box Omgevingen

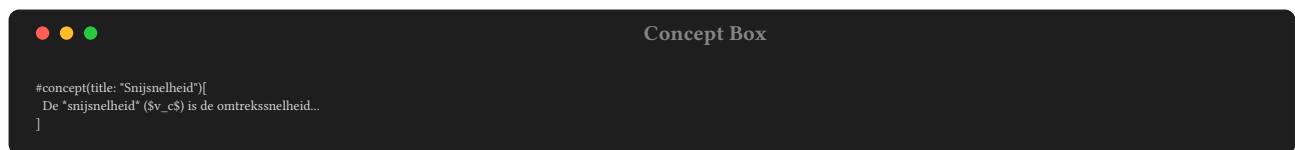
De template heeft macros om belangrijke dingen te markeren zoals definities, theorema's, oefeningen, voorbeelden en waarschuwingen. Elke box heeft een gekleurd titel-tabblad bovenaan.

7.1. Concept (Definities)

Snij snelheid

De **snij snelheid** (v_c) is de omtrekssnelheid van het gereedschap ten opzichte van het werkstuk, uitgedrukt in m/min.

Code:



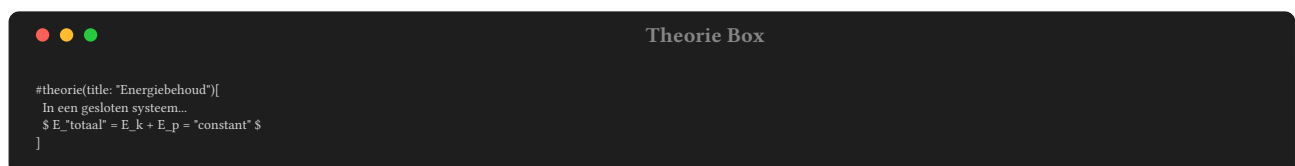
7.2. Theorie (Stellingen)

Energiebehoud

In een gesloten systeem zonder externe krachten blijft de totale mechanische energie behouden:

$$E_{\text{totaal}} = E_k + E_p = \text{constant}$$

Code:



7.3. Oefening (Oefeningen)

Oefening 1.1

Gegeven: $m = 5 \text{ kg}$, $v = 10 \text{ m/s}$

Gevraagd: Bereken de kinetische energie.

Oplossing:

$$E_k = \frac{1}{2}mv^2 = \frac{1}{2} \times 5 \times 10^2 = 250 \text{ J}$$

7.4. Voorbeeld (Praktijkvoorbeelden)

Praktisch voorbeeld: Fietser

Een fietser met massa 80 kg rijdt met 25 km/h. De kinetische energie is:

$$v = 25 \text{ km/h} = 6.94 \text{ m/s}$$

$$E_k = \frac{1}{2} \times 80 \times 6.94^2 = 1930 \text{ J} \approx 1.9 \text{ kJ}$$

7.5. Waarschuwing (Warningbox)

Veelgemaakte fout

Let op: Vergeet niet om eenheden te converteren naar SI voordat je berekeningen maakt!

7.6. Theorem

Stelling van Pythagoras

In een rechthoekige driehoek geldt: $a^2 + b^2 = c^2$

7.7. Alle box types overzicht

Type	Functie	Kleur
Theorie	#theorie(title: "...") [...]	Blauw
Concept	#concept(title: "...") [...]	Teal

Type	Functie	Kleur
Voorbeeld	#voorbeeld(title: "...") [...]	Groen
Oefening	#oefening(title: "...") [...]	Groen
Waarschuwing	#waarschuwing(title: "...") [...]	Rood
Formule	#form(title: "...") [...]	Oranje
Theorem	#theorem(title: "...") [...]	Diepblauw

8. Formules met #frm

De #frm() functie is de kern van het formularium-systeem. Het:

1. Toont de formule in een mooie oranje box
2. Registreert de formule automatisch voor het formularium

8.1. Gebruik

Σ Kinetische Energie

$$E_k = \frac{1}{2}mv^2$$

met m = massa (kg) en v = snelheid (m/s)

Code:

```

Formularium Entry

#frm("Kinetische Energie", $E_k = frac(1, 2) m v^2$, [
  met $m$ = massa ($kg$) en $v$ = snelheid ($m/s$)
])

```

8.2. Meer voorbeelden

Σ Potentiële Energie

$$E_p = mgh$$

met g = valversnelling (m/s²) en h = hoogte (m)

Σ Arbeid

$$W = \vec{F} \cdot \vec{s} = Fs \cos \theta$$

met F = kracht (N), s = verplaatsing (m), θ = hoek (rad)

Σ Meerdere Formules

$$F = ma$$

$$E = mc^2$$

Voorbeeld van meerdere formules in één blok.

8.3. Text wrap rond figuren

Gebruik #wrap-figure om tekst rond een afbeelding te laten lopen.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam erat volutpat. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensiva et.



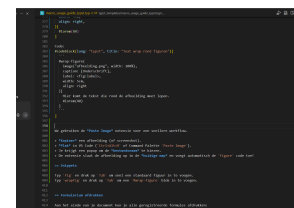
shutterstock.com · 1037719192

Figuur 1: Een wireframe placeholder.

Code:

```
#wrap-figure(
  image("afbeelding.png", width: 100%),
  caption: [Onderschrift],
  label: <fig:label>,
  width: 5cm,
  align: right
)[
  Hier komt de tekst die rond de afbeelding moet lopen.
#lorem(40)
]
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam erat volutpat. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.



Figuur 2: test

9. Afbeeldingen Toevoegen (Nieuwe Workflow)

We gebruiken de **Paste Image** extensie voor een snellere workflow.

1. **Kopieer** een afbeelding (of screenshot).
2. **Plak** in VS Code (Ctrl+Alt+V of Command Palette Paste Image).
3. Je krijgt een popup om de **bestandsnaam** te kiezen.

4. De extensie slaat de afbeelding op in de **huidige map** en voegt automatisch de `figure` code toe!

9.1. Snippets

Typ `fig` en druk op `Tab` om snel een standaard figuur in te voegen. Typ `wrapfig` en druk op `Tab` om een `#wrap-figure` blok in te voegen.

9.2. Formularium afdrukken

Aan het einde van je document kun je alle geregistreerde formules afdrukken:

```

// Eén kolom:
#printformularium()

// Twee kolommen (compact):
#printformulariumtwocol()

```

10. Symbolen met #sym

De `#sym()` functie introduceert symbolen. Bij eerste gebruik wordt een volledige uitleg getoond; daarna alleen het symbool.

10.1. Eerste vermelding

F — Kracht	N
v — Snelheid	m/s

10.2. Herhaald gebruik

Na de eerste vermelding toont `#sym(...)` alleen het symbool: F

Dit voorkomt herhaling en houdt je document overzichtelijk.

Code:

```

// Eerste gebruik (toont definitie):
#sym($F$, "Kracht", "N")

// Herhaald gebruik (toont alleen F):
#sym($F$, "Kracht", "N")

```

10.3. Symbolenlijst afdrukken

```

Symbolen Printen

```

```
// Aan het einde van je document:
#printsymbols()
```

11. Wiskunde in Typst

Typst heeft een krachtige ingebouwde wiskundemodus. De syntax is wat anders dan LaTeX maar vaak eenvoudiger.

11.1. Basis wiskunde

```

Wiskunde Basis

// Inline wiskunde:
De formule  $E = mc^2$  is beroemd.

// Display wiskunde (gecentreerd):

$$E = mc^2$$


// Breuken:

$$\frac{a}{b}$$


// Wortels:
 $\sqrt{x}$  en  $\sqrt[3]{x}$ 

```

Resultaat:

De formule $E = mc^2$ is beroemd.

$$E = mc^2$$

$$\frac{a}{b} \quad \sqrt{x} \quad \sqrt[3]{x}$$

11.2. Verzamelingen (Math Helpers)

De template bevat snelkoppelingen voor verzamelingen:

N Z Q R C

Code: `$#NN$ $#ZZ$ $#QQ$ $#RR$ $#CC$`

11.3. Afgeleiden (physica pakket)

De template bevat het **physica** pakket voor wiskundige notatie in de natuurkunde. Dit geeft je LaTeX-achtige commando's voor afgeleiden:

Code	Resultaat
<code>\$dv(y,x)\$</code>	$\frac{dy}{dx}$
<code>\$dv(y,x,2)\$</code>	$\frac{d^2y}{dx^2}$
<code>\$pdv(f,x)\$</code>	$\frac{\partial f}{\partial x}$
<code>\$pdv(f,x,2)\$</code>	$\frac{\partial^2 f}{\partial x^2}$
<code>\$grad f\$</code>	∇f
<code>\$div va(F)\$</code>	$\nabla \cdot \vec{F}$

Code	Resultaat
<code>\$curl va(F)\$</code>	$\nabla \times \vec{F}$
<code>\$laplacian f\$</code>	$\nabla^2 f$
<code>\$braket(psi, phi)\$</code>	$\langle \psi \varphi \rangle$

11.4. Matrices en Vectoren

Matrices

```
// Matrix:  
$ mat(1, 2; 3, 4) $  
  
// Vector:  
$ vec(x, y, z) $  
  
// Pijl-vector:  
$ arrow(F) $
```

Resultaat:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad \begin{pmatrix} x \\ y \\ z \end{pmatrix} \quad \vec{F}$$

11.5. Stelsels en Uitlijning

Gebruik `cases` voor stelsels en `&` voor uitlijning.

Stelsels & Uitlijning

```
// Gevalonderscheid (Cases):  
$ f(x) = cases(  
  x *als* x > 0,  
  -x *als* x <= 0  
) $  
  
// Uitlijnen op het is-gelijk-teken:  
$ a &= b + c \  
  &= d + e $
```

Resultaat:

$$f(x) = \begin{cases} x \text{ als } x > 0 \\ -x \text{ als } x \leq 0 \end{cases}$$
$$a = b + c$$
$$= d + e$$

11.6. Veelgebruikte symbolen

11.6.1. Griekse letters

Code	Res	Code	Res
<code>\$alpha\$</code>	α	<code>\$beta\$</code>	β

Code	Res	Code	Res
<code>\$gamma\$</code>	γ	<code>\$delta\$</code>	δ
<code>\$epsilon\$</code>	ε	<code>\$theta\$</code>	θ
<code>\$lambda\$</code>	λ	<code>\$mu\$</code>	μ
<code>\$pi\$</code>	π	<code>\$rho\$</code>	ρ
<code>\$sigma\$</code>	σ	<code>\$tau\$</code>	τ
<code>\$phi\$</code>	φ	<code>\$omega\$</code>	ω
<code>\$Delta\$</code>	Δ	<code>\$Omega\$</code>	Ω

11.6.2. Pijlen

Naam	Code	Symbool
Rechts	<code>\$arrow.r\$</code>	$\rightarrow$
Links	<code>\$arrow.l\$</code>	$\leftarrow$
Links & Rechts	<code>\$arrow.lr\$</code>	$\leftrightarrow$
Impliceert	<code>\$=>\$ of <code>\$implies\$</code></code>	$\Rightarrow$
Equivalent	<code>\$<==>\$ of <code>\$iff\$</code></code>	$\Leftrightarrow$
Mapsto	<code>\$ ->\$</code>	$\mapsto$
Omhoog	<code>\$arrow.t\$</code>	$\uparrow$
Omlaag	<code>\$arrow.b\$</code>	$\downarrow$

11.6.3. Operatoren en Relaties

Beschrijving	Code	Symbool
Vermenigvuldigen	<code>\$times\$</code> of <code>\$dot\$</code>	$\times$ of $\cdot$
Kleiner/Groter gelijk	<code>\$<=\$</code> / <code>\$>=\$</code>	$\leq$ / $\geq$
Ongeveer	<code>\$approx\$</code>	$\approx$
Niet gelijk	<code>\$!=\$</code> of <code>\$eq.not\$</code>	$\neq$
Evenredig	<code>\$prop\$</code>	$\propto$
Oneindig	<code>\$infinity\$</code>	∞
Nabla	<code>\$nabla\$</code>	∇
Partieel	<code>\$partial\$</code>	∂
Som	<code>\$sum\$</code>	$\sum$
Integraal	<code>\$integral\$</code>	$\int$

11.6.4. Logica en Verzamelingen

Beschrijving	Code	Symbool
Voor alle	<code>\$forall\$</code>	$\forall$
Er bestaat	<code>\$exists\$</code>	$\exists$
Element van	<code>\$in\$</code>	$\in$
Niet element	<code>\$in.not\$</code>	$\notin$

Beschrijving	Code	Symbool
Deelverzameling	<code>\$subset\$</code>	$\subset$
Vereniging	<code>\$union\$</code>	$\cup$
Doorsnede	<code>\$sect\$</code>	$\cap$
Ledge verzameling	<code>\$emptyset\$</code>	$\emptyset$

⚠ EXAMENTIP: Gebruik `$=>$` en `$<==>$` voor logische redeneringen. Deze voegen automatisch de juiste witruimte toe.

12. Cross-References

In Typst gebruik je `<label>` en `@label` voor cross-referenties:

```

Cross-References

= Mijn Sectie <sec:intro>

Later kun je verwijzen naar @sec:intro.

// Figuren:
#figure(
  image("afbeelding.png", width: 80%),
  caption: [Onderschrift],
) <fig:mijn-figuur>

Zie @fig:mijn-figuur.

```

⚠ Labels in Typst

In Typst staan labels **na** het element (niet met `\label{}` zoals in LaTeX). Labels worden geschreven als `<label-naam>` en referenties als `@label-naam`.

13. Figuren

13.1. Standaard Figuur

```

Figuur

#figure(
  image("afbeelding.png", width: 80%),
  caption: [Beschrijving van de afbeelding],
) <fig:label>

```

13.2. Figuur naast tekst

In Typst gebruik je `grid` voor layouts met tekst naast een figuur:



14. Tabellen

Typst heeft ingebouwde tabellen die eenvoudiger zijn dan LaTeX:



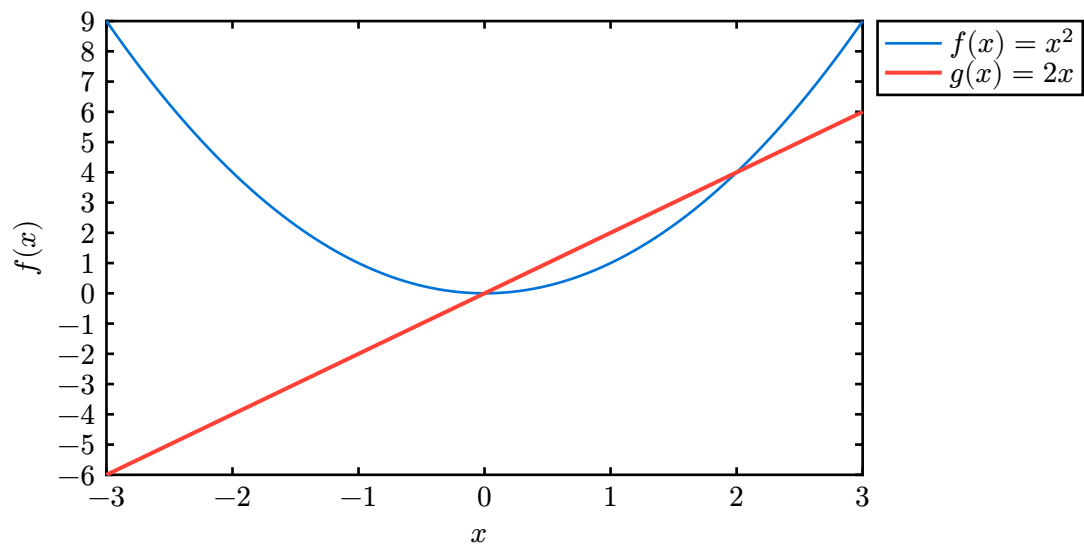
Resultaat:

Materiaal	Dichtheid	E-modulus
Staal	7850 kg/m ³	210 GPa
Aluminium	2700 kg/m ³	70 GPa
Koper	8960 kg/m ³	120 GPa

15. Grafieken en Plots (cetz-plot)

De template bevat het **cetz-plot** pakket — vergelijkbaar met `pgfplots` in LaTeX. Hiermee kun je functiegrafieken, scatterplots, staafdiagrammen en meer maken.

15.1. Functieplot



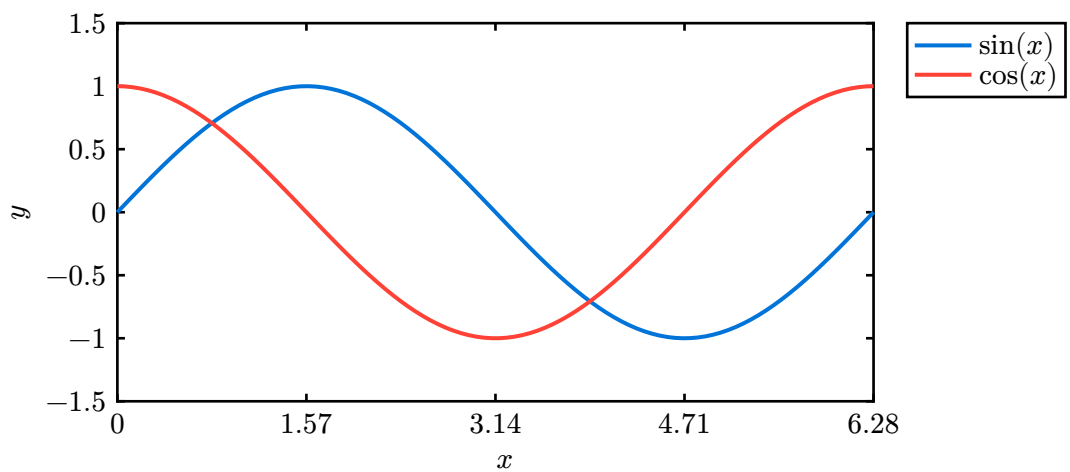
Code:

```

#cezt.canvas({
  import cetz.draw: *
  plot-plot(
    size: (10, 6),
    x-label:  $x$ , y-label:  $f(x)$ ,
    x-tick-step: 1, y-tick-step: 1,
    {
      plot.add(
        domain: (-3, 3), samples: 100,
        label:  $f(x) = x^2$ ,
        x => calc.pow(x, 2),
      )
      plot.add(
        domain: (-3, 3), samples: 100,
        label:  $g(x) = 2x$ ,
        style: (stroke: red + 1.5pt),
        x => 2 * x,
      )
    }
  )
})

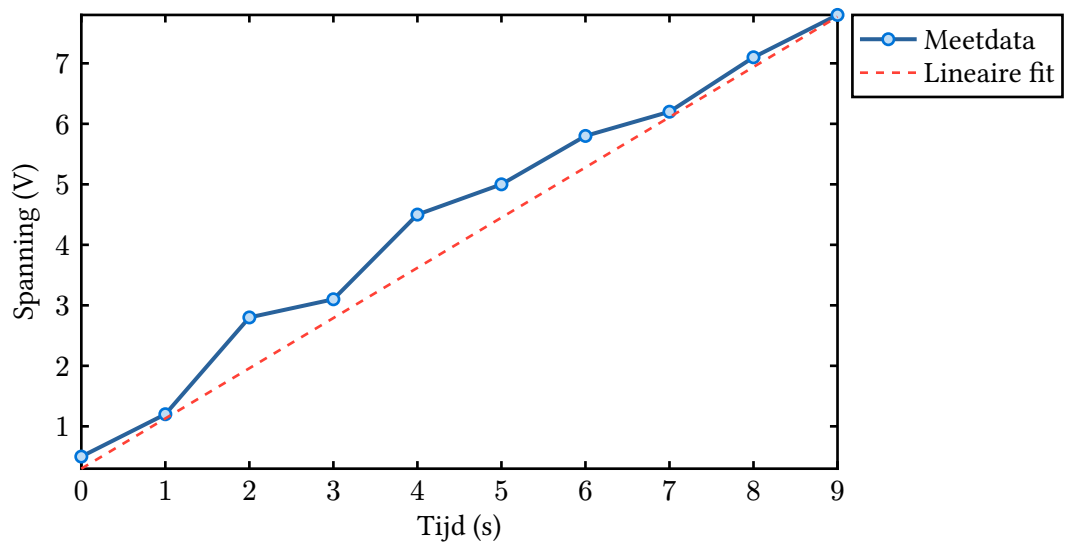
```

15.2. Trigonometrische functies



15.3. Scatterplot (data plotten)

Je kunt ook discrete datapunten plotten:

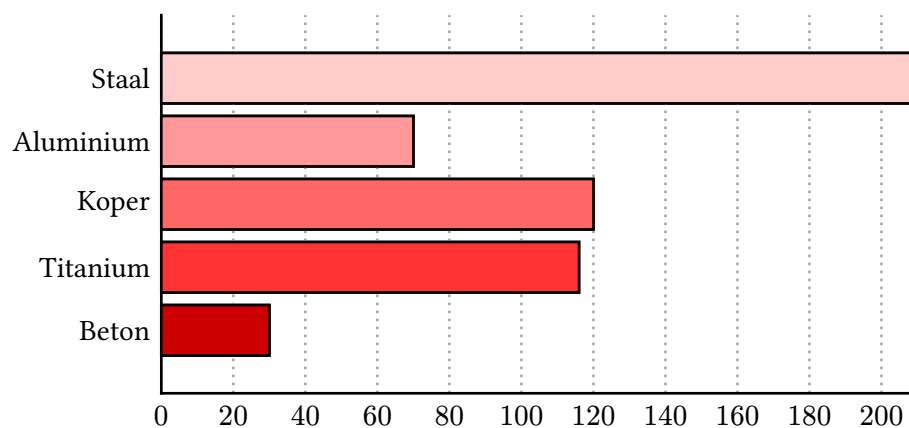


Code:

```
Scatterplot

#align(center)[
#cezt.canvas({
import cetz.draw: *
plot.plot(
size: (10, 6),
x-label: [Tijd (s)], y-label: [Spanning (V)],
{
// Discrete datapunten:
plot.add(mark: "o", mark-size: 0.15, (
(0, 0.5), (1, 1.2), (2, 2.8), ...
))
// Curve door data:
plot.add(
domain: (0, 9), samples: 50,
style: (stroke: stroke(paint: red, thickness: 1pt, dash: "dashed")),
x => 0.83 * x + 0.3,
)
}
})
]
```

15.4. Staafdiagram (Bar chart)



Code:

```

#align(center)[
#cezt.canvas{
import cetz.draw: *
chart.barchart(
size: (10, 5),
label-key: 0, value-key: 1,
(
("Staal", 210),
("Aluminium", 70),
("Koper", 120),
),
)
})
]

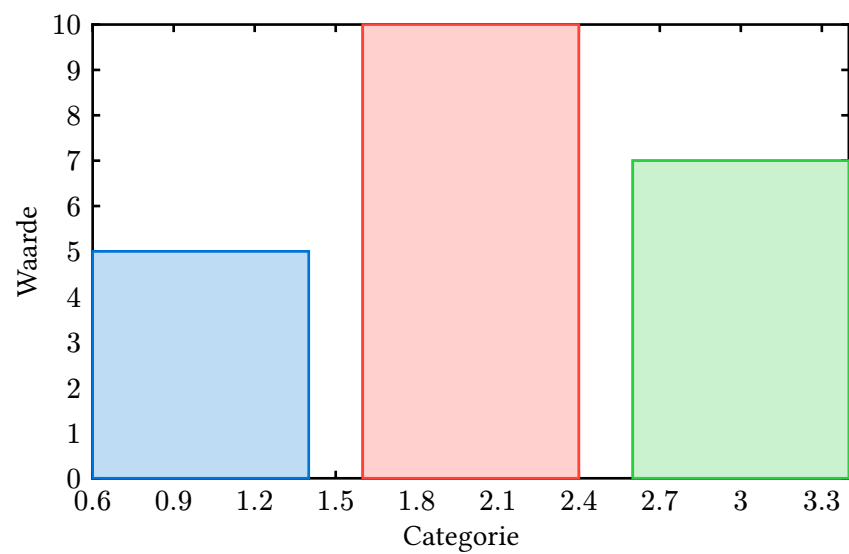
```

16. Extra Grafieken (Pie & Histogram)

Met `cetz-plot` kun je verschillende grafieken maken, waaronder pie charts.

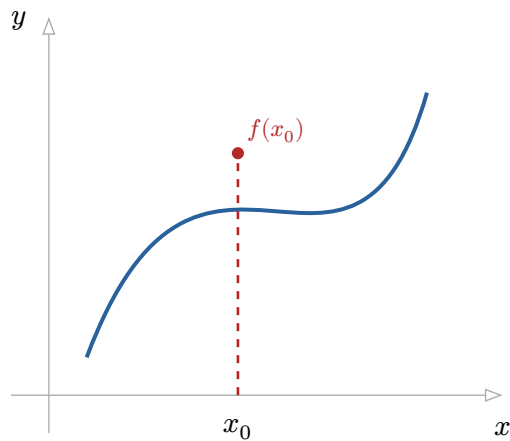
[TODO: Add pie chart]

16.1. Histogram (Bar Chart)



16.2. CeTZ Tekeningen (TikZ equivalent)

Naast plots kun je met `CeTZ` ook vrije tekeningen maken:



17. Code Listings

Gebruik de `#codeblock()` functie voor VS Code-achtige code blokken:

```
Python Script

import numpy as np

def bereken_rms(signaal):
    """Bereken Root Mean Square."""
    return np.sqrt(np.mean(signaal**2))
```

Code:

```
Codeblock Syntax

#codeblock(lang: 'python', title: 'Python Script')[
python import numpy as np def bereken_rms(signaal): return np.sqrt(np.mean(signaal**2))
]
```

18. Veelgemaakte Fouten

⚠ Vermijd deze fouten!

Fout ✗	Correct ✓
\$ E = mc^2 \$ (geen spatie)	\$ E = m c^2 \$
50 m/s (platte tekst)	\$50 "m/s"\$
Vergeten * voor vet	*vetgedrukt*
\n voor nieuwe regel	\ voor nieuwe regel
Label vóór element	Label na element: <label>

19. Utility Macro's

19.1. Workflow markers

De template bevat handige markers voor je workflow:

[**TODO: Dit moet nog worden aangevuld**] [**FIXME: Hier zit een fout**] [**NOTE: Onthoud dit**] [**CITATIE NODIG**]

Code:

```
Workflow Markers

#TODO("Dit moet nog worden aangevuld")
#FIXME("Hier zit een fout")
#NOTE("Onthoud dit")
#citeme
```

19.2. Examentip

⚠ **EXAMENTIP:** Gebruik *altijd* SI-eenheden in je antwoord op het examen!

Code: #examenbox[Gebruik *altijd* SI-eenheden...]

19.3. Belangrijke termen markeren

Gebruik #keyterm() om belangrijke vakbegrippen te markeren:

Dit is een **belangrijk concept** in de theorie.

Code: #keyterm[belangrijk concept]

19.4. Kolommen

Gebruik Typst's ingebouwde columns() voor meerdere kolommen:

```
Kolommen

#columns(2)[
  Links kolom tekst hier...

  #colbreak()

  Rechts kolom tekst hier...
]
```

20. LaTeX vs Typst Vergelijking

Een snelle referentie voor wie van LaTeX overstapt:

Feature	LaTeX	Typst
Vet	<code>\textbf{tekst}</code>	<code>*tekst*</code>

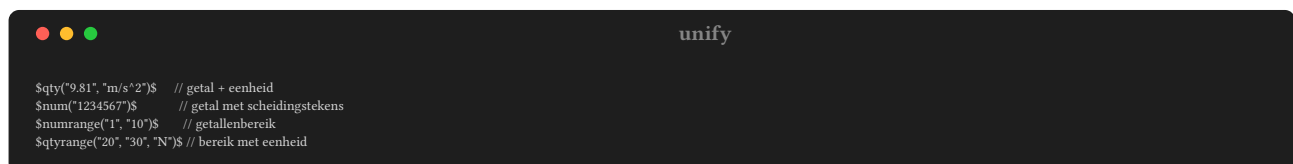
Feature	LaTeX	Typst
Cursief	<code>\textit{tekst}</code>	<code>_tekst_</code>
Heading	<code>\section{Titel}</code>	<code>= Titel</code>
Sub-heading	<code>\subsection{Titel}</code>	<code>== Titel</code>
Wiskundemodus	<code>\$E = mc^2\$</code>	<code>\$E = m c^2\$</code>
Display math	<code>\[E = mc^2 \]</code>	<code>\$ E = m c^2 \$</code>
Breuk	<code>\frac{a}{b}</code>	<code>frac(a, b)</code>
Label	<code>\label{sec:intro}</code>	<code><sec:intro></code>
Referentie	<code>\ref{sec:intro}</code>	<code>@sec:intro</code>
Lijst (bullets)	<code>\begin{itemize}...</code>	<code>- item</code>
Lijst (nummers)	<code>\begin{enumerate}...</code>	<code>+ item</code>
Afbeelding	<code>\includegraphics{...}</code>	<code>image("...")</code>
Tabel	<code>\begin{tabular}...</code>	<code>#table(...)</code>
Commentaar	<code>% commentaar</code>	<code>// commentaar</code>
Nieuwe pagina	<code>\newpage</code>	<code>#pagebreak()</code>

21. Inbegrepen Pakketten (Typst Universe)

De school-template importeert automatisch enkele essentiële pakketten van [Typst Universe](#). Je hoeft ze niet apart te importeren.

21.1. unify (siunitx equivalent)

Voor correcte opmaak van getallen, eenheden en bereiken:



Beschikbare functies

Functie	Beschrijving
<code>num("...")</code>	Getal met scheidingstekens en onzekerheden
<code>qty("...", "...")</code>	Getal met eenheid
<code>numrange("...", "...")</code>	Getallenbereik (bv. 1 tot 10)
<code>qtyrange("...", "...", "...")</code>	Bereik met eenheid

21.2. physica (physics math)

Voor wiskundige notatie in de fysica en engineering:

```

physica

$dv(y, x)$ // gewone afgeleide dy/dx
$dv(y, x, 2)$ // tweede afgeleide
$pdv(f, x)$ // partiële afgeleide
$grad f$ // gradiënt
$div va(F)$ // divergentie
$curl va(F)$ // rotatie
$laplacian f$ // Laplaciaan
$braket(psi, phi)$ // Dirac braket
$va(F)$ // vector (bold+arrow)

```



Belangrijkste physica functies

Functie	Beschrijving
$dv(y, x)$	Afgeleide $\frac{dy}{dx}$
$pdv(f, x)$	Partiële afgeleide $\frac{\partial f}{\partial x}$
$grad$	Gradiënt ∇
div	Divergentie
$curl$	Rotatie
$laplacian$	Laplaciaan ∇^2
$va(F)$	Vector met pijl $\vec{F}$
$vu(n)$	Eenheidsvector $\hat{n}$
$braket(a, b)$	Dirac braket $\langle a b \rangle$
$ketbra(a, b)$	Ket-bra $ a\rangle\langle b $
$expval(A)$	Verwachtingswaarde $\langle A \rangle$

21.3. CeTZ (TikZ equivalent)

Voor tekeningen en diagrammen, vergelijkbaar met TikZ in LaTeX:

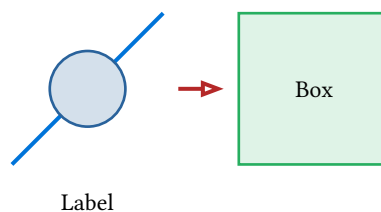
```

CeTZ Basis

#import "@preview/cetz:0.4.2"
#cezt.canvas({
  import cetz.draw: *
  // Lijn
  line((0, 0), (2, 2), stroke: blue)
  // Cirkel
  circle((1, 1), radius: 0.5, fill: red.lighten(80%))
  // Tekst
  content((1, -0.5), [Label])
})

```

Resultaat:



⚠ CeTZ is anders dan TikZ

CeTZ is geïnspireerd door TikZ maar heeft een **andere syntax**. Je kunt geen TikZ-code direct kopiëren. Raadpleeg de [CeTZ documentatie](#) voor details.

21.4. Andere handige pakketten

Deze pakketten zijn **niet** standaard inbegrepen, maar kun je zelf toevoegen. Installeren doe je door ze te importeren — Typst downloadt ze automatisch:

Extra pakket toevoegen

```
// Bovenaan je document:
#import "@preview/fletcher:0.5.8": diagram, node, edge
#import "@preview/algo:0.3.6": algo, i, d, comment, code
#import "@preview/glossarium:0.5.10": make-glossary, register-glossary, print-glossary, gls, glspl
#import "@preview/codly:1.3.0": *
#import "@preview/tablem:0.2.0": *
```

Pakket	LaTeX equivalent	Beschrijving
fletcher	tikz-cd	Diagrammen met pijlen en nodes
algo	algorithmicx	Algoritmes en pseudocode
glossarium	glossaries	Woordenlijst / afkortingenlijst
codly	minted/listings	Mooie code blokken met lijnummers
tablem	—	Markdown-achtige tabellen
chemformula	mhchem	Chemische formules en reacties
alchemist	chemfig	Structuurformules tekenen

21.4.1. Fletcher — Diagrammen met pijlen

Fletcher maakt het eenvoudig om flowcharts, toestandsdiagrammen en commutatieve diagrammen te tekenen.

Fletcher Flowchart

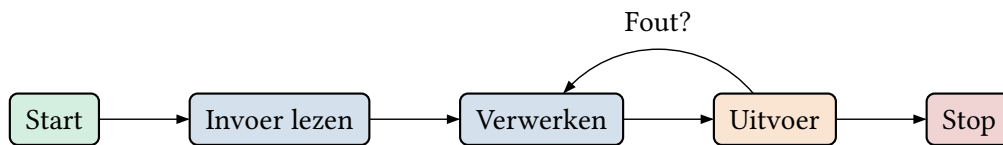
```
#import "@preview/fletcher:0.5.8" as fletcher: diagram, node, edge

#diagram(
  node-stroke: 0.8pt,
  node-corner-radius: 3pt,
  spacing: (12mm, 10mm),

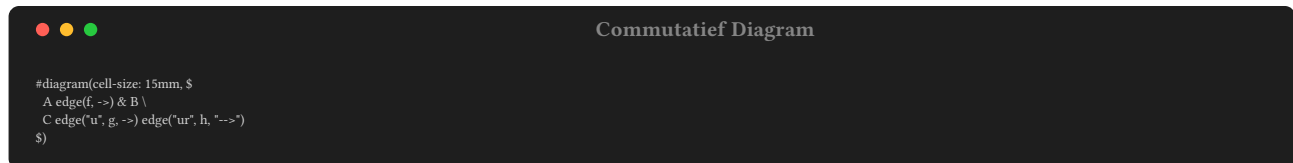
  node((0,0), [Start], fill: schoolGreen.lighten(80%)),
  edge("-|>"),
  node((1,0), [Invoer lezen], fill: schoolBlue.lighten(80%)),
  edge("-|>"),
  node((2,0), [Verwerken], fill: schoolBlue.lighten(80%)),
  edge("-|>"),
  node((3,0), [Uitvoer], fill: schoolOrange.lighten(80%)),
  edge("-|>"),
  node((4,0), [Stop], fill: schoolRed.lighten(80%)),

  // Feedback lus:
  edge((3,0), (2,0), [Fout?], "-|>", bend: -50deg),
)
```

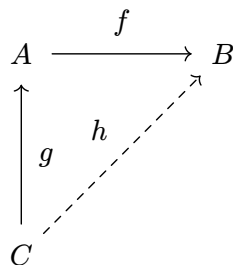
Resultaat:



Een wiskundig commutatief diagram:



Resultaat:



21.4.2. Algo – Pseudocode en Algoritmes

De algo package maakt prachtige pseudocode blokken:



Resultaat:

BUBBLESORT (A, n) :

1 **for** $j \leftarrow 0$ **to** $n - 2$: **for** $k \leftarrow 0$ **to** $n - j - 2$: **if** $A[k] > A[k + 1]$: swap $A[k]$ **and** $A[k + 1]$ **return** A

BINARYSEARCH($A, n, target$):

```

1   $l \leftarrow 0, r \leftarrow n - 1$ 
2  while  $l \leq r$ :  $m \leftarrow \lfloor \frac{l+r}{2} \rfloor$ 
3      if  $A[m] = target$ : return  $m$  if  $A[m] < target$ :  $l \leftarrow m + 1$  else:
         $r \leftarrow m - 1$  return  $-1$ 

```

// Gevonden! Niet gevonden

21.4.3. Glossarium – Woordenlijst

Met glossarium maak je een woordenlijst met afkortingen en definities:

Glossarium

```

#import "@preview/glossarium:0.5.10": make-glossary, register-glossary, print-glossary, gls, glspl

#show: make-glossary

#let entries = (
  (key: "fem", short: "FEM", long: "Finite Element Method",
   description: "Numerieke methode voor het oplossen van PDEs"),
  (key: "cfd", short: "CFD", long: "Computational Fluid Dynamics",
   description: "Numerieke simulatie van vloeistofstroming"),
  (key: "plc", short: "PLC", long: "Programmable Logic Controller",
   description: "Industriële computer voor automatisering"),
)
#register-glossary(entries)

// In je tekst:
De @fem methode wordt vaak gebruikt in combinatie met @cfd.
Bij een eerste vermelding wordt de volledige naam getoond.

// Aan het einde van je document:
#print-glossary(entries)

```

21.4.4. Tablem – Markdown-achtige tabellen

Snelle tabellen met Markdown-syntax:

Tablem

```

#import "@preview/tablem:0.2.0": tablem

#tablem[
| "Materiaal" | "$E$ (GPa)" | "$rho$ (kg/m³)" |
|-----|-----|-----|
| Staal | 210 | 7850 |
| Aluminium | 70 | 2700 |
| Titanium | 116 | 4500 |
]

```

Resultaat:

Materiaal	E (GPa)	ρ (kg/m ³)
Staal	210	7850
Aluminium	70	2700
Titanium	116	4500

22. Afbeeldingen naast elkaar

Gebruik `grid()` of `figure(grid(...))` om afbeeldingen naast elkaar te zetten.



shutterstock.com · 1037719192



shutterstock.com · 1037719192

Figuur 3: Twee afbeeldingen naast elkaar.

Code:

```
Side-by-side

#figure(
  grid(
    columns: (1fr, 1fr),
    gutter: 1em,
    image("left.png", width: 100%),
    text: [Hier kun je ook tekst zetten bijvoorbeeld.],
    // of nog een afbeelding:
    // image("right.png", width: 100%),
  ),
  caption: [Twee afbeeldingen naast elkaar.],
  label: <fig:side-by-side>
)
```

23. Customisatie (Set & Show Rules)

Met `set` pas je eigenschappen aan, met `show` verander je hoe iets eruit ziet.

23.1. Set Rules

Gebruik `set` voor standaardwaarden (font, grootte, kleur).

```
Set Rules

#set text(font: "Arial", size: 12pt)
#set par(justify: true)
#set list(marker: [--])
```

23.2. Show Rules

Gebruik `show` om elementen volledig te herdefiniëren.

```
Show Rules

// Verander alle "Typst" woorden in vetgedrukt blauw
#show "Typst": name => text(blue, weight: "bold")[#name]

// Kader rond alle afbeeldingen
#show image: it => box(stroke: 2pt + red, it)
```

```
// Pas headings aan
#show heading.where(level: 1): it => [
  #set text(fill: schoolBlue)
  #block(it)
  #line(length: 100%, stroke: 1pt + schoolBlue)
]
```

24. Scripting & Automatisatie

Typst is een volwaardige programmeertaal.

24.1. Variabelen en Functies

```
#let naam = "Typst"
#let optellen(a, b) = a + b

Hallo #naam!
1 + 2 = #optellen(1, 2)
```

Vars & Funcs

24.2. Conditionals (If/Else)

```
#let score = 15
Resultaat: #if score >= 10 [Geslaagd] else [Niet geslaagd]
```

If/Else

24.3. Lussen (Loops)

Je kan itereren over arrays of getallen.

```
#let kleuren = (red, orange, yellow, green, blue)

#for kleur in kleuren [
  #text(fill: kleur)■
]

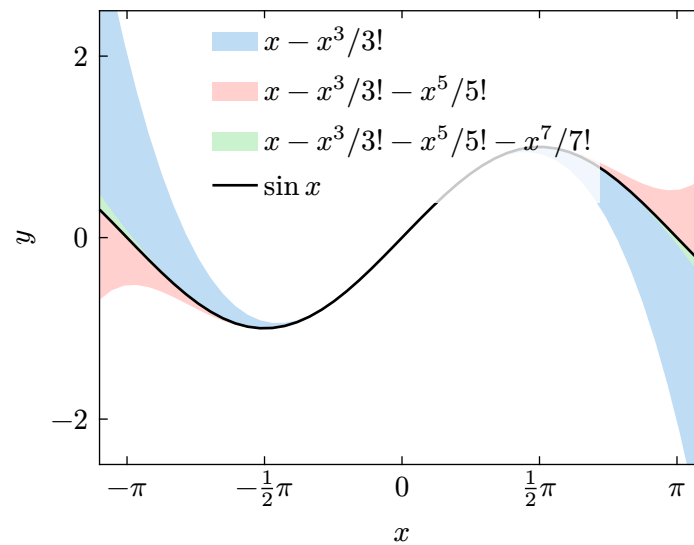
// Native vormen (geen Cetz nodig):
#for i in range(5) [
  #circle(radius: 2pt + i*1pt, fill: black.lighten(i*20%))
]
```

Loops

25. Geavanceerde CeTZ Voorbeelden

Hier zijn enkele geavanceerde voorbeelden van wat mogelijk is met CeTZ.

25.1. Taylor Reeks (Fill Between)



Code:

```

Taylor Reeks

#cetz.canvas({
  import cetz.draw: *
  let f1(x) = calc.sin(x)
  let fn = (
    ($ x - x^3/3! $, x => x - calc.pow(x, 3)/6),
    // ...
  )

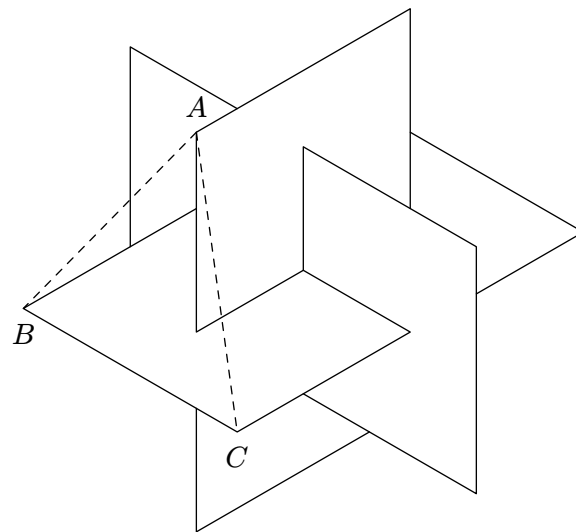
  set-style(axes: (stroke: .5pt, tick: (stroke: .5pt)),
    legend: (stroke: none, orientation: ttb, item: (spacing: .3), scale: 80%))

  plot.plot(size: (8, 6),
    x-tick-step: calc.pi/2,
    y-tick-step: 2, y-min: -2.5, y-max: 2.5,
    legend: "inner-north",
    {
      let domain = (-1.1 * calc.pi, +1.1 * calc.pi)

      for ((title, f)) in fn {
        plot.add-fill-between(f, f1, domain: domain,
          style: (stroke: none), label: title)
      }
      plot.add(f1, domain: domain, label: $ sin x $,
        style: (stroke: black))
    })
})

```

25.2. 3D Orthogonale Projectie



Code:

```

3D Ortho

#cetz.canvas(length: 2cm, {
  import cetz.draw: *
  let phi = (1 + calc.sqrt(5)) / 2

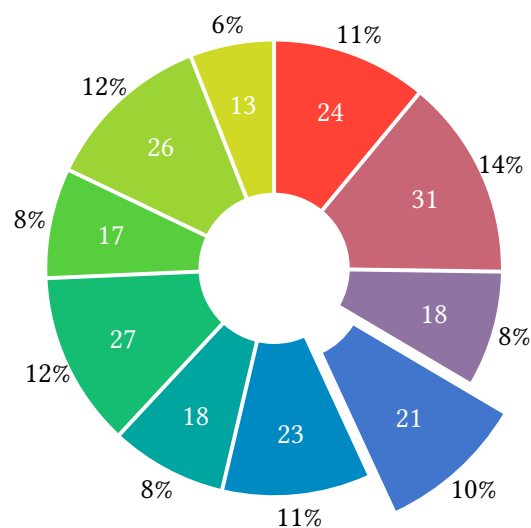
  ortho({
    // Definieer vlakken (verborgen)
    hide({
      line((-phi, -1, 0), (-phi, 1, 0), (phi, 1, 0), (phi, -1, 0), close: true, name: "xy")
      // ...
    })

    // Bereken intersecties
    intersections("a", "yz", "xy")
    // ...

    // Teken lijnen
    set-style(stroke: (thickness: 0.5pt, cap: "round", join: "round"))
    line((0, 0, 0), "c.1", (phi, 1, 0), (phi, -1, 0), "c.3")
    // ...
  })
})

```

25.3. Pie Chart (Landen)



Code:

```

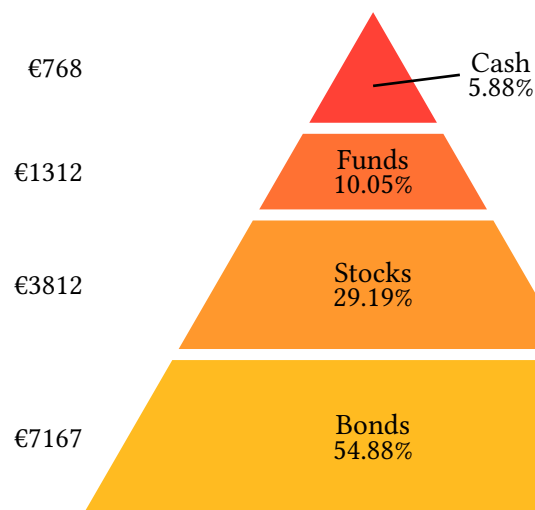
Pie Chart

#cetz.canvas({
  let colors = gradient.linear(red, blue, green, yellow)
  let data = (
    ([Belgium], 24),
    ([Germany], 31),
    ([Greece], 18),
    // ...
  )

  chart.piechart(
    data,
    value-key: 1,
    label-key: none,
    radius: 4,
    stroke: none,
    slice-style: colors,
    inner-radius: 1,
    outset: 3,
    inner-label: (content: (value, label) => [#text(white, str(value))], radius: 110%),
    outer-label: (content: "%", radius: 110%)
  )
})

```

25.4. Pyramid Chart (Financieel)



Code:

```

Pyramid Chart

#cetz.canvas({
  let colors = gradient.linear(red, yellow)
  let data = (
    ([Cash], 768),
    ([Funds], 1312),
    ([Stocks], 3812),
    ([Bonds], 7167),
  )
  let total = data.map(i => i.last()).sum()

  chart.pyramid(
    data,
    value-key: 1,
    label-key: 0,
    mode: 'AREA-HEIGHT',
    stroke: none,
    level-style: colors,
    inner-label: (
      content: (value, label) => align(center, stack(
        label + "n",
        str(calc.round(value / total * 10000) / 100) + "%",
        spacing: 2pt,
        dir: ttb
      ))
    )
  )
})

```

```
),  
// ...  
gap: 10%  
)  
}
```

Symbolenlijst

Symbool	Beschrijving	Eenheid	Pagina
F	Kracht	N	16
v	Snelheid	m/s	16

Formularium

Kinetische Energie

[p.14](#)

$$E_k = \frac{1}{2}mv^2$$

met m = massa (kg) en v = snelheid (m/s)

Potentiële Energie

[p.14](#)

$$E_p = mgh$$

met g = valversnelling (m/s²) en h = hoogte (m)

Arbeid

[p.14](#)

$$W = \vec{F} \cdot \vec{s} = Fs \cos \theta$$

met F = kracht (N), s = verplaatsing (m), θ = hoek (rad)

Meerdere Formules

[p.15](#)

$$F = ma$$

$$E = mc^2$$

Voorbeeld van meerdere formules in één blok.